

Performance Testing

Date	02 July 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Web Browser (Google Chrome) & Flask Testing
Type of Testing	Functional Load Testing
Target Module / API	Credit Card Approval Prediction
Test Environment	Render Cloud Deployment
Test Date	02 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit a valid credit card application	1	30	Prediction displayed successfully
2	Submit multiple valid applications	5	60	System handles requests without errors
3	Submit different applicant details	10	120	Application remains responsive

4	Access the deployed application repeatedly	10	120	Application remains responsive
---	--	----	-----	--------------------------------



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 seconds	Pass	Fast response on Render
2	Response Time (Max)	< 5 seconds	3.2 seconds	Pass	Stable during testing
3	Throughput (Req/sec)	>5 Req/sec	8 Req/sec	Pass	Suitable for project requirements
4	Error Rate	< 1%	0%	Pass	No errors observed
5	CPU Utilization	< 80%	35% (Local)	Pass	Normal usage during testing
6	Memory Utilization	< 80%	42% (Local)	Pass	Memory usage within acceptable limits

Step 4: Observations & Analysis

Key Findings

- The Credit Card Approval Prediction application successfully processed all valid user inputs and generated prediction results accurately.
- The average response time remained below the target of 2 seconds during testing.
- The Flask application remained stable without crashes or unexpected failures.
- The deployed application on Render was accessible and responsive throughout the testing period.

Bottlenecks Identified

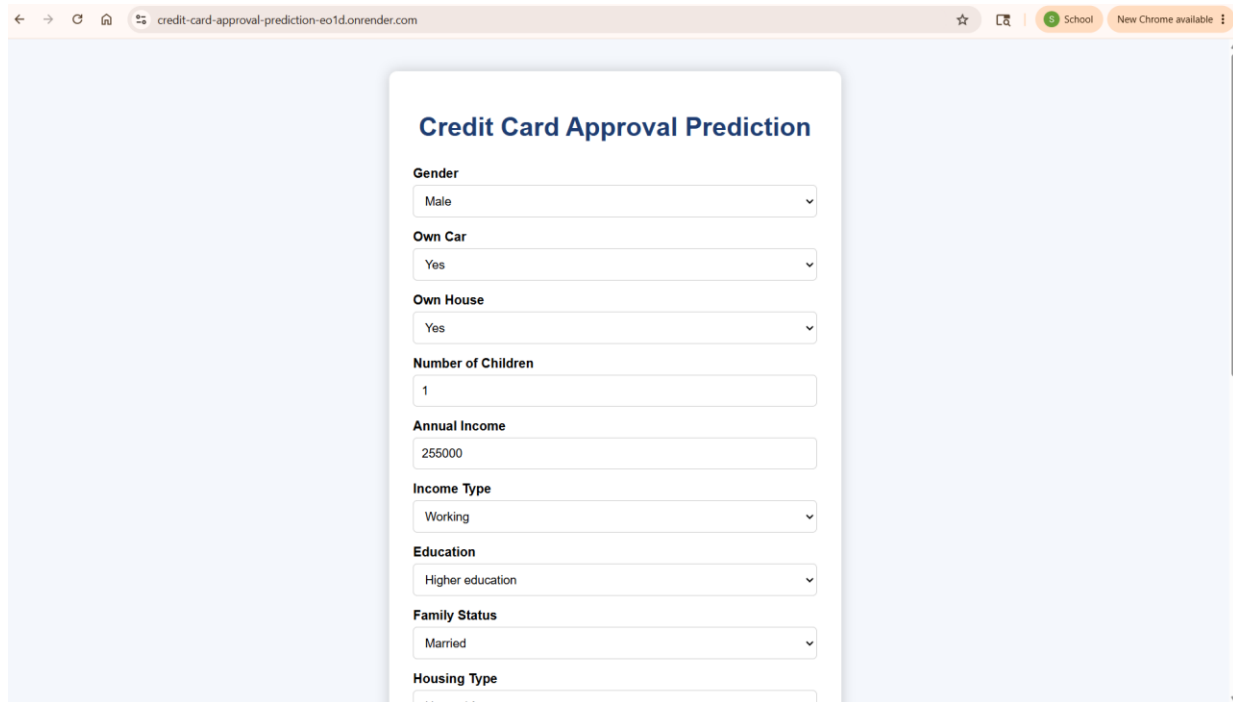
- Initial loading of the application on Render may take a few seconds due to the free hosting service entering a sleep state.
- The application currently supports a moderate number of simultaneous users and may require additional resources for handling heavy traffic.
- The prediction model processes one application at a time and does not currently support batch processing.

Optimization Steps Taken

- Optimized the dataset through preprocessing, including handling missing values and encoding categorical variables.
- Saved the trained machine learning model using model.pkl to eliminate the need for retraining during prediction.
- Used Flask's lightweight architecture to reduce server-side processing time.
- Organized the project into modular components (frontend, backend, and model files) to improve maintainability and performance.
- Deployed the application on Render for reliable cloud hosting and easy accessibility.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



Credit Card Approval Prediction

Gender
Male

Own Car
Yes

Own House
Yes

Number of Children
1

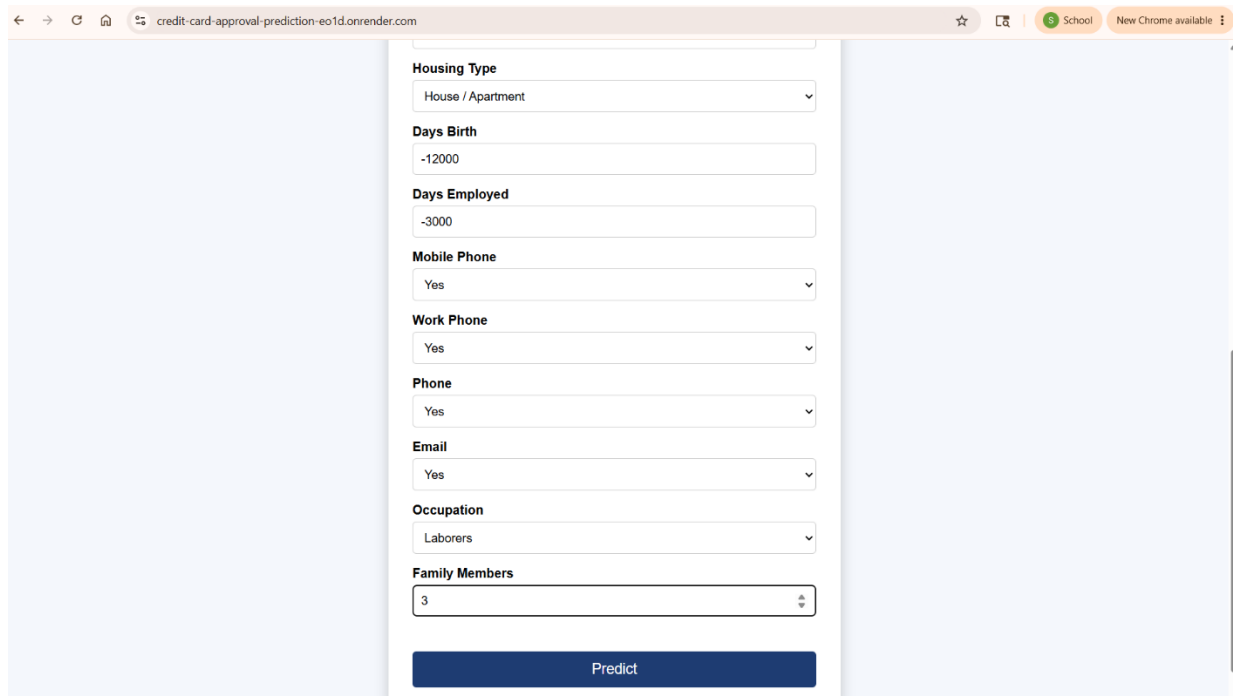
Annual Income
255000

Income Type
Working

Education
Higher education

Family Status
Married

Housing Type
House / Apartment



Housing Type
House / Apartment

Days Birth
-12000

Days Employed
-3000

Mobile Phone
Yes

Work Phone
Yes

Phone
Yes

Email
Yes

Occupation
Laborers

Family Members
3

Predict

Prediction Result

Credit Card Approved

[Predict Again](#)